Name : Bayu Khaldun Ardiansyah

Student ID : F1d02510043

Source Github Repository

1. Overview

This project implements a singly linked list data structure in Java. The `LinkedList` class inherits from the `AbstractList` abstract class and acts as a container to store various types of data using the `Object` type. There is a base class named `Entity` which is extended into two specific subclasses, `Anime` and `Digimon`. Through this design, a single linked list implementation can hold dynamic objects, demonstrating the principles of Abstract Data Types (ADT) and inheritance in Java.

Project Structure

File	Role
`Entity.java`	Base class that stores data as an `Object` and provides basic methods (like `toString`).
`LinkedList.java`	Contains the `Node` class (list element), `AbstractList` (ADT blueprint), and `LinkedList` (concrete implementation of *insert*, *deleteTail*, and *display* operations).
`MainNode.java`	Contains the `Anime` and `Digimon` classes (subclasses of `Entity`) and the `MainNode` class which serves as the main program to test all list operations.

How to Run

```
javac *.java
java MainNode
```

Data Structure Illustration

The figure below illustrates the content of the list in `MainNode` after 4 items are added and the `deleteTail` operation is executed. `head` points to the first node, while iteration is used to reach the element right before the last one to remove the tail.

```
head
  ↓
Node(Anime:"Kakkoi")  →  Node(Digimon:"Fly")  →  Node(Anime:"Kawaii")  →  null
```

2. Entity.java

The `Entity` class serves as a generic data entity representation that can store any data type by utilizing the `Object` class.

Entity.java, lines 1-6: attributes and constructor

```
public class Entity {
    Object data;

    public Entity(Object data){
        this.data = data;
    }
```

- `public class Entity`: Declares a public class named `Entity`.
- `Object data`: A class variable/attribute to hold the data (using the `Object` type so it can accept any data type).
- `public Entity(Object data)`: The constructor of the `Entity` class that takes an `Object` type argument.
- `this.data = data;`: Assigns the `data` argument passed through the constructor parameter to the instance's `data` attribute.

Entity.java, lines 8-12: toString method

```
    @Override
    public String toString() {
        return String.valueOf(this.data);
    }
}
```

- `@Override`: An annotation indicating that the method below overrides a method from its superclass (in this case, the built-in Java `Object` class).
- `public String toString()`: A method that converts the object's representation into a String type.
- `return String.valueOf(this.data);`: Returns the String representation of the value in the `data` attribute. This is called automatically when the `Entity` object is printed.
- `}`: Closes the `toString` method and the `Entity` class.

3. LinkedList.java

This file defines three parts: `Node` as the single list element, `AbstractList` as the list's blueprint, and `LinkedList` as the concrete implementation.

3.1 Node Class

LinkedList.java, lines 1-9: Node attributes and constructor

```
class Node {
    Object data;
    Node next;

    Node(Object data) {
```

```
        this.data = data;
        this.next = null;
    }
}
```

- `class Node`: Declares a package-private `Node` class that acts as the building block of the linked list.
- `Object data`: An attribute to store the actual data inside the node.
- `Node next`: A pointer attribute that stores a reference to the next `Node` object in the list.
- `Node(Object data)`: The `Node` constructor called when a new node is created.
- `this.data = data;`: Stores the `data` argument value into this object's `data` attribute.
- `this.next = null;`: Initializes the `next` pointer to `null`, as every new node does not point to any node yet.
- `}`: Closes the constructor and the `Node` class.

3.2 AbstractList Class

LinkedList.java, lines 11-18: Abstract List declaration

```
abstract class AbstractList {
    Node head;
    Node tail;

    public abstract void insert(Object data);
    public abstract void deleteTail();
    public abstract void display();
}
```

- `abstract class AbstractList`: Declares an abstract class that becomes the blueprint/rules for the derived list classes.
- `Node head;`: Declares a pointer that will always point to the first element in the list.
- `Node tail;`: Declares a pointer that will point to the last element in the list.
- `public abstract void insert(Object data);`: Defines an abstract method (without implementation) for inserting new data.
- `public abstract void deleteTail();`: Defines an abstract method for deleting the tail element (last element).
- `public abstract void display();`: Defines an abstract method for displaying all list contents.
- `}`: Closes the abstract class.

3.3 LinkedList Class & insert()

LinkedList.java, lines 20-34: LinkedList implementation and insert()

```
public class LinkedList extends AbstractList {

    @Override
    public void insert(Object data) {
        Node newNode = new Node(data);
```

```
        if (head == null) {
            head = newNode;
            tail = newNode;
            return;
        }

        tail.next = newNode;
        tail = newNode;
    }
```

- `public class LinkedList extends AbstractList`: Declares the concrete `LinkedList` class that inherits the structure from `AbstractList`.
- `@Override`: An annotation indicating this is the implementation of the abstract method from its superclass.
- `public void insert(Object data)`: Implementation of the method to add a node to the back of the list (insert at tail).
- `Node newNode = new Node(data);`: Creates a new `Node` object containing the passed `data`.
- `if (head == null) {`: Checks if the list is still empty (no elements yet).
- `head = newNode;` and `tail = newNode;`: If empty, both the `head` and `tail` pointers are directed to this `newNode`.
- `return;`: Exits the method, ending the insert process.
- `tail.next = newNode;`: If the list is not empty, the current last element (`tail`) points (`next`) to the new node.
- `tail = newNode;`: Updates the `tail` pointer to refer to `newNode` as the new last node.
- `}`: Closes the `insert` method.

3.4 deleteTail()

LinkedList.java, lines 36-52: deleteTail implementation

```
    @Override
    public void deleteTail() {
        if (head == null) {
            return;
        }

        if (head.next == null) {
            head = null;
            return;
        }

        Node current = head;
        while(current.next.next != null) {
            current = current.next;
        }
```

```
            current.next = null;
        }
```

- `@Override` & `public void deleteTail()`: Defines the implementation to remove the very last node (tail).
- `if (head == null) { return; }`: Empty List Case. If there is no head, there is nothing to delete, and the method stops immediately.
- `if (head.next == null) { head = null; return; }`: Single Element Case. If the list only has one node (the node after `head` is `null`), then `head` is set to `null` making the list empty, and the method stops.
- `Node current = head;`: Creates a temporary pointer named `current` starting from the front (head).
- `while(current.next.next != null) {`: Loops to find the node **before the last node**. The loop continues as long as "the node after the next node" is not `null`.
- `current = current.next;`: Advances the `current` pointer to the next node continuously until it reaches one position before the last node.
- `current.next = null;`: Removes the reference to the last node by setting the `next` pointer at this position to `null`. (Note: in this implementation, `tail` itself is not updated, although ideally it should be updated to point to `current`).
- `}`: Closes the `deleteTail` method.

3.5 display()

LinkedList.java, lines 54-65: display implementation

```
    @Override
    public void display() {
        Node tail = head;

        while (tail != null) {
            System.out.print(tail.data + "->");
            tail = tail.next;
        }

        System.out.println("null");
    }
}
```

- `@Override` & `public void display()`: Implementation of the method to print the entire content of the linked list.
- `Node tail = head;`: Declares a temporary variable named `tail` (not the class variable `tail`) that takes the starting point `head` to perform the traversal iteration.
- `while (tail != null) {`: Starts a loop; the condition is that the iteration continues as long as `tail` (the temporary pointer) is not empty (null).
- `System.out.print(tail.data + "->");`: Prints the `data` content of the node to the terminal/console followed by an arrow `"->"`. Because `print` is used, the printing happens on the same line. Here, the data's `toString` method is implicitly called.
- `tail = tail.next;`: Advances the temporary pointer to the next node.

- `System.out.println("null");`: After the loop finishes (reaching the end of the list), it prints the text `"null"` at the end of the list, followed by a newline from `println`.
- `}`: Closes the `display` method and the `LinkedList` class implementation.

4. MainNode.java

This file contains two subclasses of `Entity` (namely `Anime` and `Digimon`), and `MainNode` which acts as the main driver program.

4.1 Anime & Digimon Class

MainNode.java, lines 1-17: Anime and Digimon subclasses

```
class Anime extends Entity {
    public Anime(Object data) {
        super(data);
    }

    public void Genre(Object data) {
        System.out.println(data);
    }
}

class Digimon extends Entity {
    public Digimon(Object data) {
        super(data);
    }

    public void Aksi(Object data) {
        System.out.println(data);
    }
}
```

- `class Anime extends Entity {`: Declares the `Anime` subclass that inherits properties from the parent class `Entity`.
- `public Anime(Object data) { super(data); }`: Constructor of the `Anime` class. The `super(data)` command is used to call the parent class's constructor (`Entity`) to initialize the `data` value.
- `public void Genre(Object data) { System.out.println(data); }`: A unique method in the `Anime` class named `Genre` that receives an `Object` value and prints it directly to the console.
- `class Digimon extends Entity {`: Declares the `Digimon` subclass which also inherits properties from the parent class `Entity`.
- `public Digimon(Object data) { super(data); }`: Constructor of the `Digimon` class that calls the superclass `Entity`'s constructor.
- `public void Aksi(Object data) { System.out.println(data); }`: A unique method in the `Digimon` class named `Aksi` that will print its data directly to the terminal screen.

4.2 Main Method

MainNode.java, lines 19-30: List instantiation and data insertion

```
public class MainNode {
    public static void main(String[] args) {
        LinkedList list = new LinkedList();

        Anime aku = new Anime("Digimon");

        Digimon kau = new Digimon("Omnimon");

        list.insert(new Anime("Kakkoi"));
        list.insert(new Digimon("Fly"));
        list.insert(new Anime("Kawaii"));
        list.insert(new Digimon("Roar"));
```

- `public class MainNode`: Declares the main class containing the execution method.
- `public static void main(String[] args)`: The entry point method where the Java application starts running.
- `LinkedList list = new LinkedList();`: Creates an object or instance of the linked list data structure that we will use.
- `Anime aku = new Anime("Digimon");`: Initializes an object of type `Anime` with the variable name `aku` and the initial value `"Digimon"`.
- `Digimon kau = new Digimon("Omnimon");`: Initializes an object of type `Digimon` with the variable name `kau` and the initial value `"Omnimon"`.
- `list.insert(...)`: Performs the process of adding new data elements (which are direct instantiations of the `Anime` and `Digimon` types) to the tail of the list sequentially. This proves that this container is generic (capable of holding both `Anime` and `Digimon` types because both are *Objects / Entities*).

MainNode.java, lines 31-39: deleting data and displaying the list

```
        list.deleteTail();

        System.out.println("Linked list: ");
        list.display();
        aku.Genre("Tanoshii");
        kau.Aksi("Blast");

    }
}
```

- `list.deleteTail();`: Calls the method to remove the data at the very last position of the list. The node containing the element "Roar" is deleted from the list's memory at this stage.
- `System.out.println("Linked list: ");`: Prints the text "Linked list: " to the console.
- `list.display();`: Displays the entire contents of the linked list on the screen after the previous tail deletion operation.
- `aku.Genre("Tanoshii");`: Calls the specific `Genre` method of the `aku` object (of type `Anime`) by passing the String argument `"Tanoshii"`, which this method will directly print to the console.

- `kau.Aksi("Blast");`: Calls the specific `Aksi` method of the `kau` object (of type `Digimon`) by passing the String argument `"Blast"`, which will also be printed.
- `}`: Curly brace closing the execution of the `main` method.
- `}`: Curly brace closing the scope block of the `MainNode` class.

5. Program Output

Here is the output of the program when executed based on the `MainNode.java` file above:

```
Linked list:
Kakkoi->Fly->Kawaii->null
Tanoshii
Blast
```

Output Explanation:

- The list initially has 4 items (Kakkoi, Fly, Kawaii, Roar). After the `deleteTail()` instruction is executed, "Roar" is removed from the list's memory.
- Then, the `list.display()` method prints the contents of the list and prints `null` at the end.
- Afterwards, the call to the specific attribute `aku.Genre()` prints the text "Tanoshii".
- Finally, the call to the specific attribute `kau.Aksi()` prints the text "Blast".